Escuela de Ingeniería de Sistemas y Computación

Desarrollo de Software II

750009C

John Robert Causimanse C.
Ingeniero de sistemas
Maestría en proyectos de software de innovación

Evaluación de Factores de Calidad Estandarizados.
Jenkins

¿Cuál es el propósito de utilizar Jenkins?

Universidad del Valle

Evaluación de Factores de Calidad Estandarizados.
Jenkins

Su propósito es actuar como un inspector de control de calidad automático para tu proyecto en PHP, mediante un **JenkinsFile, ver adjunto.**

.



Universidad del Valle

Evaluación de Factores de Calidad Estandarizados.
Jenkins

JenkinsFile, este archivo, valida el flujo de trabajo automatizado, se verifica el PHP y que se pueda utilizar en el Xampp.

.

Universidad del Valle

Evaluación de Factores de Calidad Estandarizados.
Jenkins

JenkinsFile: Revisa la sintaxis de PHP, lee el texto del archivo y verifica que la gramática de PHP sea correcta (paréntesis cerrados, punto y coma, nombres de variables válidos). Si hay un error, lo reporta inmediatamente.

Universidad del Valle

DevOps y el Control de Versiones.



Universidad del Valle

DevOps y el Control de Versiones.

Si tú y otra persona intentan editar los archivos en la misma carpeta de tu computadora o enviándose archivos por correo/WhatsApp, van a sobrescribir el trabajo del otro y será un caos.

Universidad del Valle

DevOps y el Control de Versiones.

Git + un Repositorio Remoto como GitHub o GitLab.



Universidad del Valle

DevOps y el Control de Versiones.

¿Qué es Git?

Git es el control de versiones es una herramienta para gestionar los cambios en un conjunto de archivos a lo largo del tiempo. Git es un sistema de control de versiones distribuido que permite a los usuarios introducir nuevas ediciones y versiones de manera fluida, manteniendo un historial completo del código fuente



DevOps y el Control de Versiones.

Sin Git, esté es el escenario de tu local:

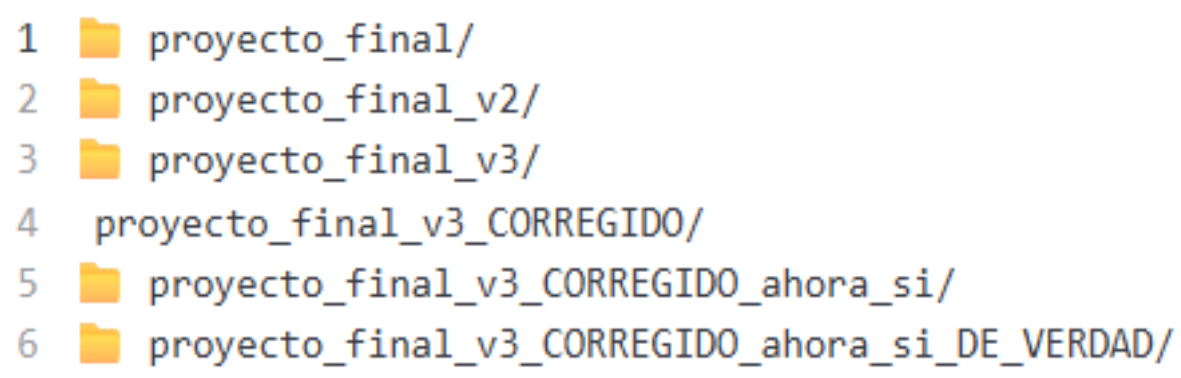
1. proyecto_final/
2. proyecto_final_v2/
3. proyecto_final_v3/
4. proyecto_final_v3_CORREGIDO/
5. proyecto_final_v3_CORREGIDO_ahora_si/
6. proyecto_final_v3_CORREGIDO_ahora_si_DE_VERDAD/

No sabes qué versión es la correcta.

Universidad del Valle

DevOps y el Control de Versiones.

Sin Git, esté es el escenario de tu local:



Si algo se rompe, no puedes volver atrás.

Universidad del Valle

DevOps y el Control de Versiones.

Sin Git, esté es el escenario de tu local:

1. proyecto_final/
2. proyecto_final_v2/
3. proyecto_final_v3/
4. proyecto_final_v3_CORREGIDO/
5. proyecto_final_v3_CORREGIDO_ahora_si/
6. proyecto_final_v3_CORREGIDO_ahora_si_DE_VERDAD/

Si dos personas editan el mismo archivo, se sobrescriben.

Universidad del Valle

DevOps y el Control de Versiones.

Sin Git, esté es el escenario de tu local:

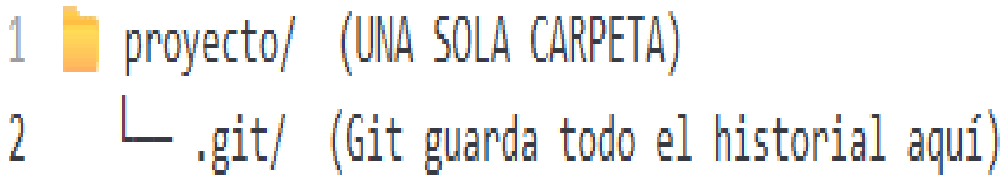
1. proyecto_final/
2. proyecto_final_v2/
3. proyecto_final_v3/
4. proyecto_final_v3_CORREGIDO/
5. proyecto_final_v3_CORREGIDO_ahora_si/
6. proyecto_final_v3_CORREGIDO_ahora_si_DE_VERDAD/

No sabes quién cambió qué cosa

Universidad del Valle

DevOps y el Control de Versiones.

Con Git

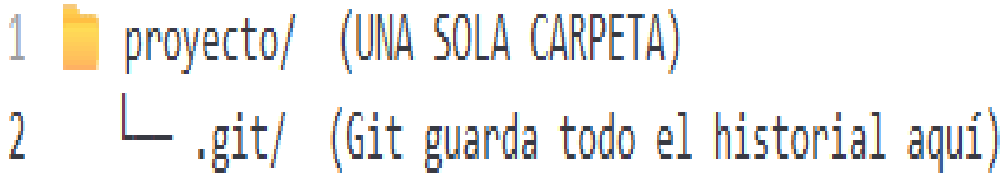


Una sola carpeta con todo el historial

Universidad del Valle

DevOps y el Control de Versiones.

Con Git

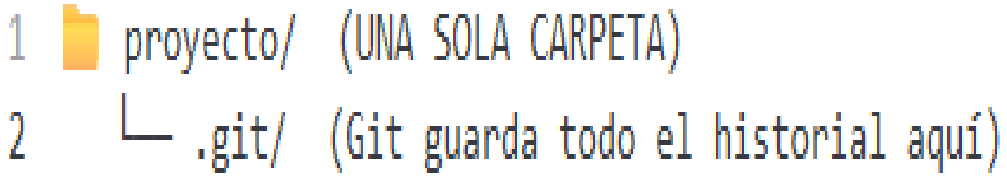


Puedes volver a cualquier versión anterior

Universidad del Valle

DevOps y el Control de Versiones.

Con Git

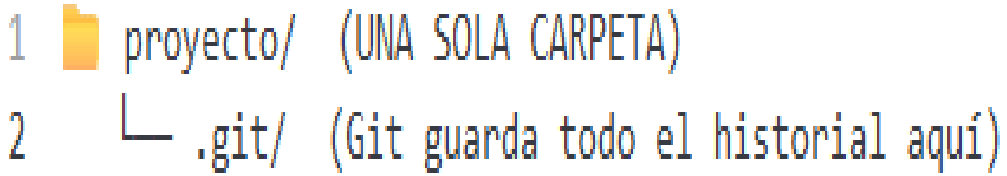


Varias personas pueden trabajar sin pisarse.

Universidad del Valle

DevOps y el Control de Versiones.

Con Git



Sabes exactamente quién cambió qué, cuándo y por qué.

Universidad del Valle

DevOps y el Control de Versiones.

Conceptos claves de Git

Repositorio (Repo):	Es la carpeta de tu proyecto donde Git guarda todo el historial. Puede estar en tu PC (local) o en internet (remoto, como GitHub).

Universidad del Valle

DevOps y el Control de Versiones.

Conceptos claves de Git

Commit: Es un punto de guardado. Cada vez que haces un commit, Git toma una "foto" de todos los cambios que hiciste y los guarda con un mensaje descriptivo.

Universidad del Valle

DevOps y el Control de Versiones.

Conceptos claves de Git

Commit: git commit -m "Agregué el formulario de login".

Universidad del Valle

DevOps y el Control de Versiones.

Conceptos claves de Git

Rama (Branch): Es una línea de tiempo alternativa. Imagina que tu código principal es una carretera (main). Puedes crear un desvío" (rama) para experimentar sin afectar la carretera principal.



DevOps y el Control de Versiones.

Conceptos claves de Git

Rama (Branch): git checkout -b feature/nuevo-modulo.



DevOps y el Control de Versiones.

Conceptos claves de Git

Push: Es subir tus commits locales a un repositorio remoto (GitHub).

Universidad del Valle

DevOps y el Control de Versiones.

Conceptos claves de Git

Push: git push origin main.



DevOps y el Control de Versiones.

Conceptos claves de Git

Pull: Es bajar los cambios que otros hicieron al repositorio remoto.

Universidad del Valle

DevOps y el Control de Versiones.

Conceptos claves de Git

Pull: git pull origin main.



DevOps y el Control de Versiones.

Conceptos claves de Git

Merge: Es unir una rama con otra. Ejemplo: unir tu rama experimental con la principal.

Universidad del Valle

Importante

"Git" y "GitHub". Son cosas diferentes:



Universidad del Valle

Git y DevOps



Universidad del Valle

Importante en GIT



Universidad del Valle

DevOps y el Control de Versiones.

1. La Única Fuente de la Verdad (Repositorio Remoto).

C:\xampp\htdocs\MiProyecto, no debería de vivir en un repositorio local, si no en un repositorio GitHub o GitLab, donde vive el codigo oficial.



DevOps y el Control de Versiones.

2. Clonar el proyecto

Los desarrolladores no te piden los archivos. Ellos van a su propia computadora, abren su terminal y descarga una copia exacta del proyecto:

git clone https://github.com/tu-usuario/comercial-pro.git

Universidad del Valle

La Única Fuente de la Verdad (Repositorio Remoto).

DevOps y el Control de Versiones.

3. Nadie trabaja directamente sobre la rama principal (main).

Regla de oro. La rama main (o master) siempre debe tener código que funcione.

Si un desarrollador va a crear una nueva función (ej: un módulo de ventas). Crea una rama nueva:

git checkout -b feature/modulo-ventas.

La Única Fuente de la Verdad (Repositorio Remoto).

DevOps y el Control de Versiones.

4. Subir cambios y pedir revisión (Pull Request)

git push origin feature/modulo-ventas.

Luego, en GitHub, crea un **Pull Request (PR)** o **Merge Request**. Esto en paralelo es como decir: *"Ya terminé el módulo de ventas, ¿alguien puede revisarlo para unirlo al código principal?"*

¿Dónde entra Jenkins en este trabajo en equipo?



¿Dónde entra Jenkins en este trabajo en equipo?

Aquí es donde tu pipeline se vuelve muy importante. En lugar de que tú o un humano revisen el código de tu compañero línea por línea, Jenkins lo hace automáticamente.

¿Dónde entra Jenkins en este trabajo en equipo?

Se configura algo llamado Webhook:

1. Un desarrollador crea el Pull Request en GitHub.

2. GitHub le envía una señal automática (Webhook) a tu Jenkins: *"¡hay código nuevo para revisar!"*

3. Jenkins se despierta solo, descarga ese código nuevo en un espacio temporal.

¿Dónde entra Jenkins en este trabajo en equipo?

1. Jenkins ejecuta tu pipeline (verifica PHP, revisa sintaxis, cuenta archivos).

2. Jenkins le reporta a GitHub:

Si todo está bien: GitHub muestra un check verde y permite unir el código a main.

Si hay un error de sintaxis: GitHub muestra una X roja y bloquea la unión del código. Le dice al compañero desarrollador: Arregla el error en la línea 42 antes de poder unir tu código.

Pongamos en practica

Paso 1: Sube tu proyecto a GitHub

1.GIT



Pongamos en practica

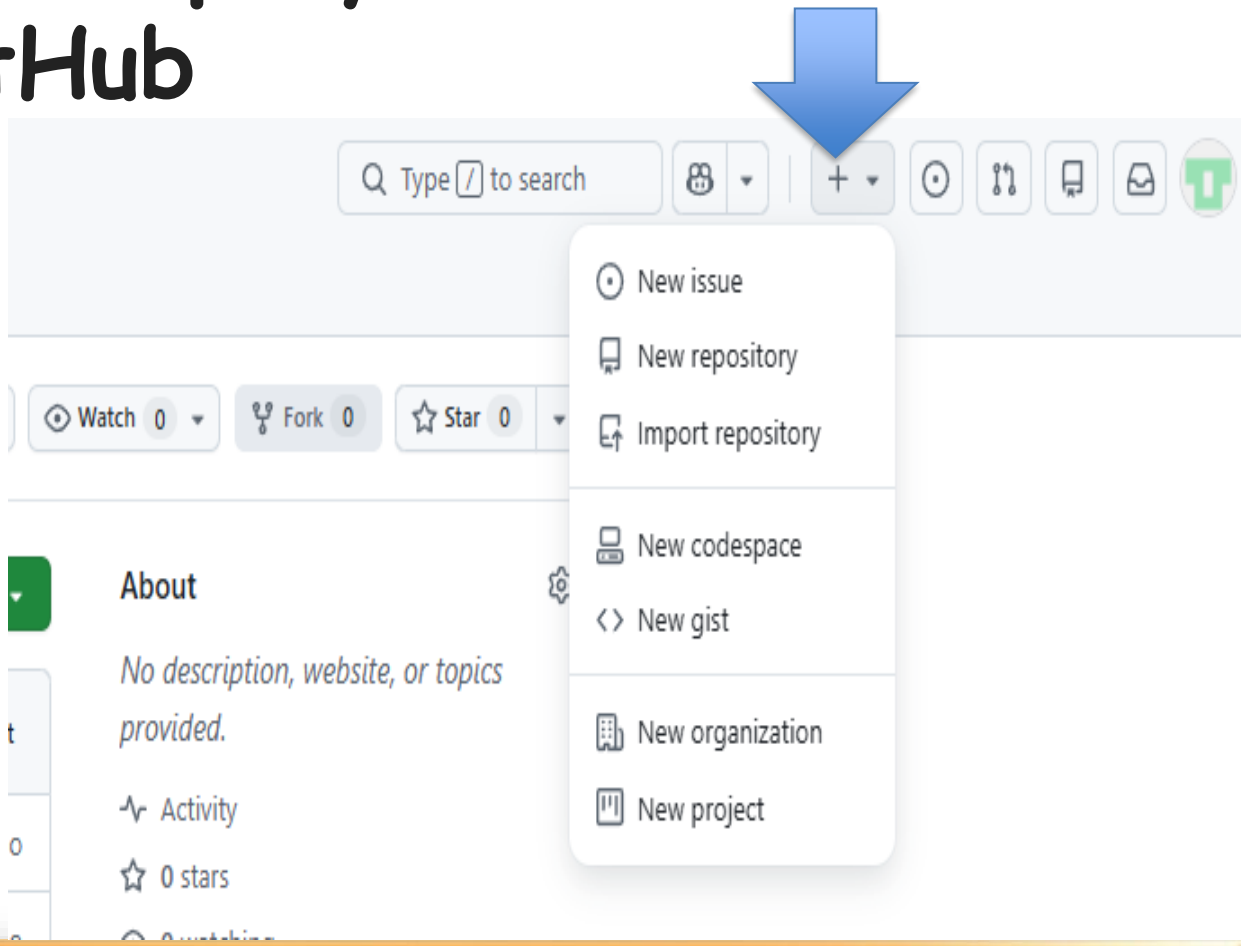
Paso 1: Sube tu proyecto a GitHub

1. Crea una cuenta en github.com.

Pongamos en practica

Paso 1: Sube tu proyecto a GitHub

2. Crea un "New Repository" llamado comercial-pro.



Pongamos en practica

3. En tu carpeta local C:\xampp\htdocs\comercial-pro, abre la terminal (o Git Bash) y ejecuta:

git init (Inicia el proyecto en git).

git add . (Agrega archivos al espacio de preparación).

Primer Commit

git commit -m "Primer commit: estructura inicial del proyecto"

Pongamos en practica

3. En tu carpeta local C:\xampp\htdocs\comercial-pro, abre la terminal (o Git Bash) y ejecuta:

Renombrar la rama a "main"

git branch -M main

Conectar con GitHub

git remote add origin https://github.com/TU_USUARIO/catalogo-ap.git

Universidad del Valle

Pongamos en practica

3. En tu carpeta local C:\xampp\htdocs\comercial-pro, abre la terminal (o Git Bash) y ejecuta:

Si todo sale bien:

```
Enumerating objects: 10, done.
Counting objects: 100% (10/10), done.
Delta compression using up to 8 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (10/10), 3.54 KiB | 1.77 MiB/s, done.
Total 10 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/jhonrot/comercial-pro.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Pongamos en practica

3. En tu carpeta local C:\xampp\htdocs\comercial-pro, abre la terminal (o Git Bash) y ejecuta:

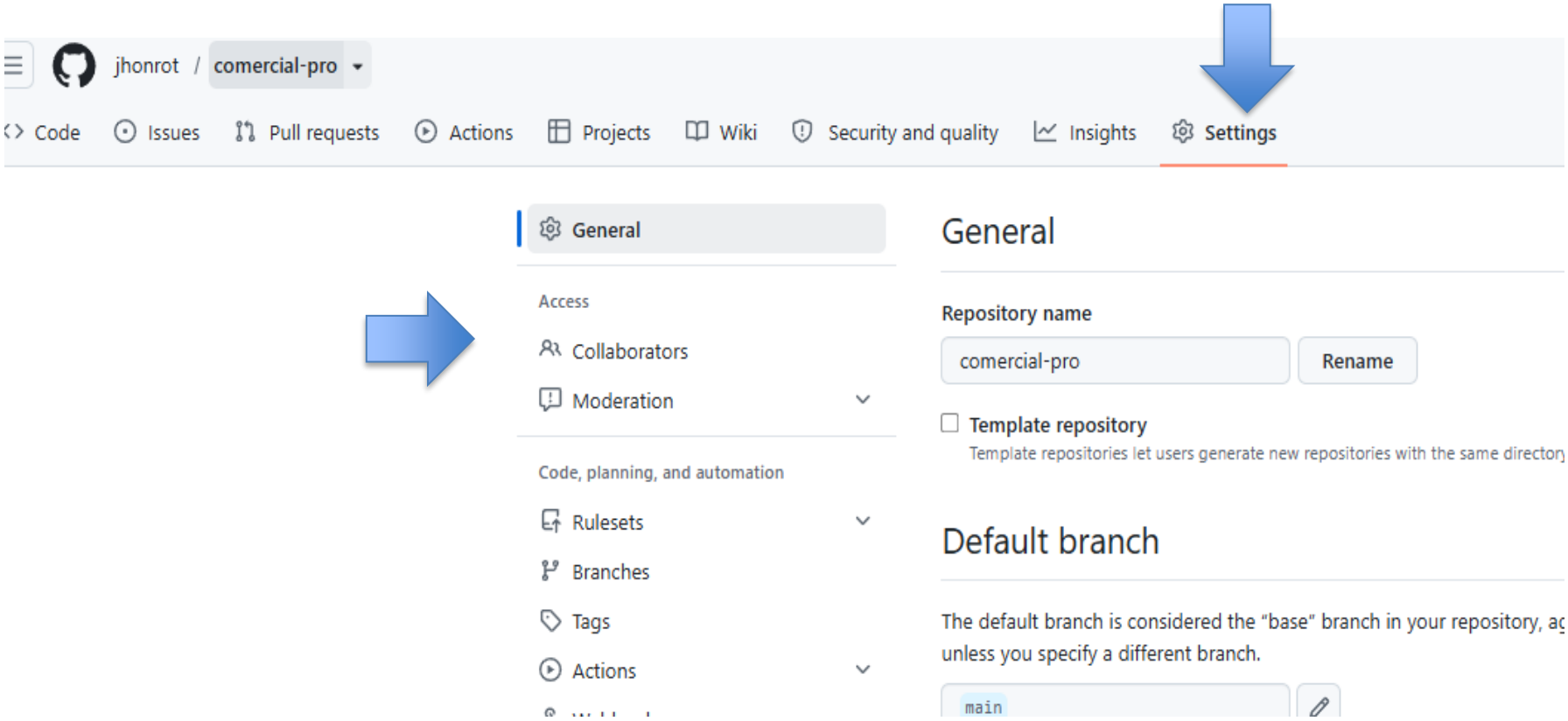
Recarga tu página de GitHub, ***verás tus archivos PHP*** *en el repositorio.*

Pongamos en practica

Paso 2: Invita a tus colaboradores

En GitHub, ve a Settings > Collaborators y agrega el correo de tu colaborador. Ahora él puede clonar el proyecto.

Pongamos en practica



Pongamos en practica

Paso 2: Tus colaboradores una vez reciban la invitación por correo y la acepte, debe clonar el proyecto:

cd C:\xampp\htdocs
git clone https://github.com/jhonrot/comercial-pro.git

Pongamos en practica

Paso 2: Y cada que hagan cambios los deben subir así:

```
# 1. Ir a la carpeta del proyecto
cd C:\xampp\htdocs\comercial-pro

# 2. Ver qué archivos cambió
git status

# 3. Agregar los cambios
git add .

# 4. Hacer un commit con descripción
git commit -m "Agregué módulo de reportes de ventas"

# 5. Subir los cambios a GitHub
git push origin main
```

Pongamos en practica

Paso 2:

Importante

Nunca se trabaje directamente en main
La rama main debe tener siempre código que funcione. Cada desarrollador debe crear su propia rama para trabajar:

Crear una rama nueva con tu nombre o la funcionalidad

```
git checkout -b feature/modulo-ventas-juan
```

Pongamos en practica

Paso 2:

Importante

Sincronizar antes de empezar a trabajar

Siempre actualizar el código con los últimos cambios con:

git pull origin main

Pongamos en practica

Paso 2:

Importante

Hacer commits pequeños y descriptivos

NO es recomendado: git commit -m "cambios".

SI es recomendado: git commit -m "Agregué validación de email en formulario de registro".

Pongamos en practica

Paso 3: Evoluciona tu Jenkinsfile (Nivel Avanzado)

Actualmente, tu Jenkins lee los archivos directamente de tu carpeta local (C:\xampp\htdocs\...).



Pongamos en practica

Paso 3: Evoluciona tu Jenkinsfile (Nivel Avanzado)

Para trabajo en equipo, cambiarás esa parte del Jenkinsfile para que diga: En lugar de leer mi carpeta local, ve a GitHub, descarga la última versión del código y revísala.

Pongamos en practica

Paso 3: Evoluciona tu Jenkinsfile (Nivel Avanzado)

Ahora para que Jenkins revise automáticamente cada cambio que sube cualquier colaborador, hacer lo siguiente en el *jenkisFile* :

Pongamos en practica

Paso 3: Evoluciona tu Jenkinsfile (Nivel Avanzado)

(ver archivo adjunto de nombre jenkisFileAutomatizado)

```
pipeline {
  agent any
   stages {
       stage('Descargar código de GitHub') {
           steps {
              // Jenkins descarga automáticamente el código del repo
              git branch: 'main', url: 'https://github.com/tu-usuario/comercial-pro.git'
            }
         }
        // ... aquí van tus etapas de revisión de PHP que ya creamos ...
      }
    }
```

Pongamos en practica

Automatización de Jenkins

Ahora los desarrolladores trabajan en equipo:

git clone https://github.com/jhonrot/comercial-pro.git

Pongamos en practica

Automatización de Jenkins

Hacer cambios y subirlos automáticamente:

```
git add .
git commit -m "Descripción del cambio"
git push
```

Roles y responsabilidades de Jenkins y GIT



Taller
Jenkins y Git (DevOps)

1. Trabajar con el proyecto Catalgo-ap y trabajarlo tanto Jenkins como en Git, integrar los dos programas.

2. Realizar la práctica en grupos de mínimo 3 y máximo 4 integrantes. Uno de ellos asumirá el rol de líder, quien se encargará de asignar y administrar la implementación de los siguientes requerimientos:

- Al diligenciar un formulario, si ocurre un error en los datos ingresados, la información introducida no debe borrarse. El sistema debe señalar claramente la ubicación del error para que el usuario pueda corregirlo y continuar. (1 pto)

Taller
Jenkins y Git (DevOps)

- El formulario incluirá un catálogo de ciudades de Colombia, Italia y Francia. Si se selecciona o ingresa una ciudad (por ejemplo, Roma), el sistema debe autocompletar automáticamente el país correspondiente (Italia). (1 pto)

- Corregir la funcionalidad de los botones "Ver detalles", asegurando que muestren la información adicional correspondiente al producto. (1 pto)

- Realizar la integración y entrega conticnua por Git y al menos una modificación que se muestre en la misma aplicación (Git) y ver el historial de integración en Jenkins. (2 ptos)